

some are GSM-based, while others use less common network standards like FOMA and TD-SCDMA. Each network operator uses a different kind of network infrastructure and this limits the flow of information.

Scripting: The variety of devices being used makes executing the test script (scripting) a key challenge. As devices differ in keystrokes, input methods, menu structure and display properties, a single script does not function on every device.

Mobile application testing tools

Using specific as well as specialised tools, mobile apps can be tested on multiple parameters and dimensions.

iPhone

The following tools can be downloaded and used to test mobile applications that run on the iPhone.

Clang Static Analyzer: URL: <http://clang-analyzer.lvm.org>

XCode: URL: https://developer.apple.com/library/prerelease/ios/documentation/ToolsLanguages/Conceptual/Xcode_Overview/UnitTestYourApp/UnitTestYourApp.html

Android

A wide range of tools can be downloaded and used for testing mobile applications that run on Android. Some of these are listed here.

Android Lint: This is integrated with the Eclipse IDE for Android app testing. This software highlights the potential bugs and performance problems in the app. The Android Lint tool is a static code analysis tool that checks the Android project source files for potential bugs. It also optimises and improves the app's performance, usability, security, accessibility and internationalisation features.

URL: <http://developer.android.com/tools/help/lint.html>

FindBugs: This is an open source code library for static analysis. It is a special plugin for Android-specific coding, and is a static code analysis tool for finding bugs in standard Java programs.

URL: <https://code.google.com/p/findbugs-for-android>

Appium: Appium is an open source test automation framework for use with native and hybrid mobile apps. It works for iOS and Android apps using the WebDriver JSON wire protocol. Appium is cross-platform and allows the writing of tests for multiple platforms (iOS, Android) using the same API. It enables code to be reused between iOS and Android test suites.

Native apps are those written using the iOS or Android SDKs. Mobile Web apps are those accessed using a mobile browser (Appium supports Safari on iOS and Chrome, or the built-in 'Browser' app on Android). Hybrid apps have a wrapper around a 'webview'—a native control that enables interaction with Web content. Projects like Phonegap make it easy to build apps using Web technologies that are then bundled into a native wrapper, creating a hybrid app.

URL: <http://www.appium.io>

Robotium: Robotium is the Android test automation framework that offers full support for native and hybrid applications. Robotium makes it easy for engineers to write powerful and robust automatic black-box UI tests for Android applications. With the use of Robotium, the test case developers can write function, system and user acceptance test scenarios, spanning multiple Android activities.

The following are the features of Robotium:

- It can test any type of Android app - native and hybrid.
- Detailed knowledge of the application under test is not required.
- The framework handles multiple activities of Android automatically.
- Less time consuming when writing solid and complex test cases.
- Readability of test cases is greatly improved, compared to standard instrumentation tests.
- Test cases are more efficient and robust due to the run-time binding to UI components.
- Speedy and accurate test case execution.
- Integrates easily with Maven, Gradle or Ant to run tests as part of continuous integration.

URL: <https://code.google.com/p/robotium>

MonkeyRunner: This tool provides an API for writing the programs that can control the Android device or emulator. Using MonkeyRunner, the test engineer can write a Python program that installs an Android application or test package, runs it, sends keystrokes, takes screenshots of its GUI, and stores the screenshots on the workstation.

The MonkeyRunner tool was developed primarily to test applications and devices at the functional or framework level and to run unit test suites, but test engineers are free to use it for other purposes.

This software allows test engineers to perform functional app testing by providing the API for managing the device. MonkeyRunner makes use of Jython, an implementation of Python that uses the Java programming language. Jython allows the MonkeyRunner API to interact directly and easily with the Android framework. With the use of Jython, the test engineer can use Python syntax to access the constants, classes, and methods of the API.

MonkeyRunner does not have advanced features like Robotium. The tool has three modules:

- **MonkeyRunner**, to manage connections to the devices. It allows users to create test scripts with Python or just record and play them.
- **MonkeyDevice**, to manage apps and events.
- **MonkeyImage**, to gather screenshots for advanced test reporting.

The MonkeyRunner tool provides the following unique features for Android testing:

- **Multiple device control:** The MonkeyRunner API can apply one or more test suites across multiple devices or